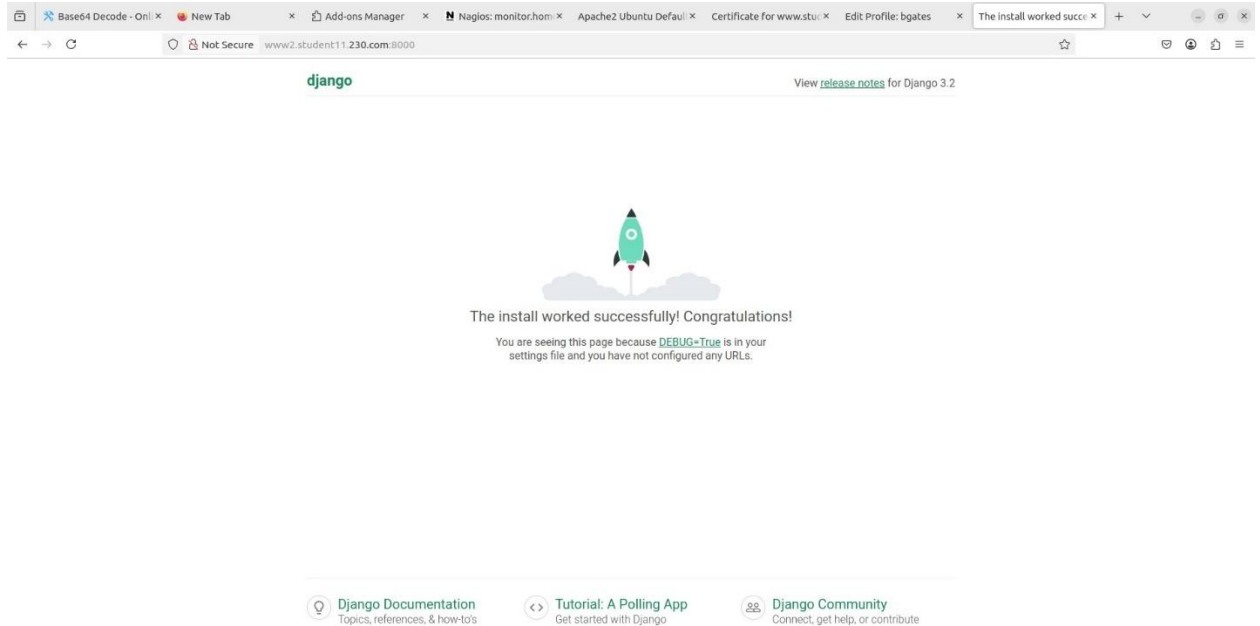
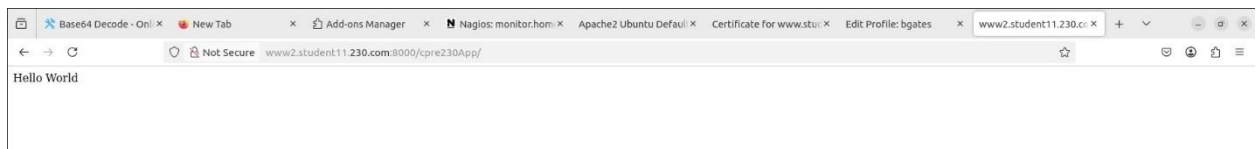


Lab 09 Template

1.) Screenshot of default Django landing page (10 points)

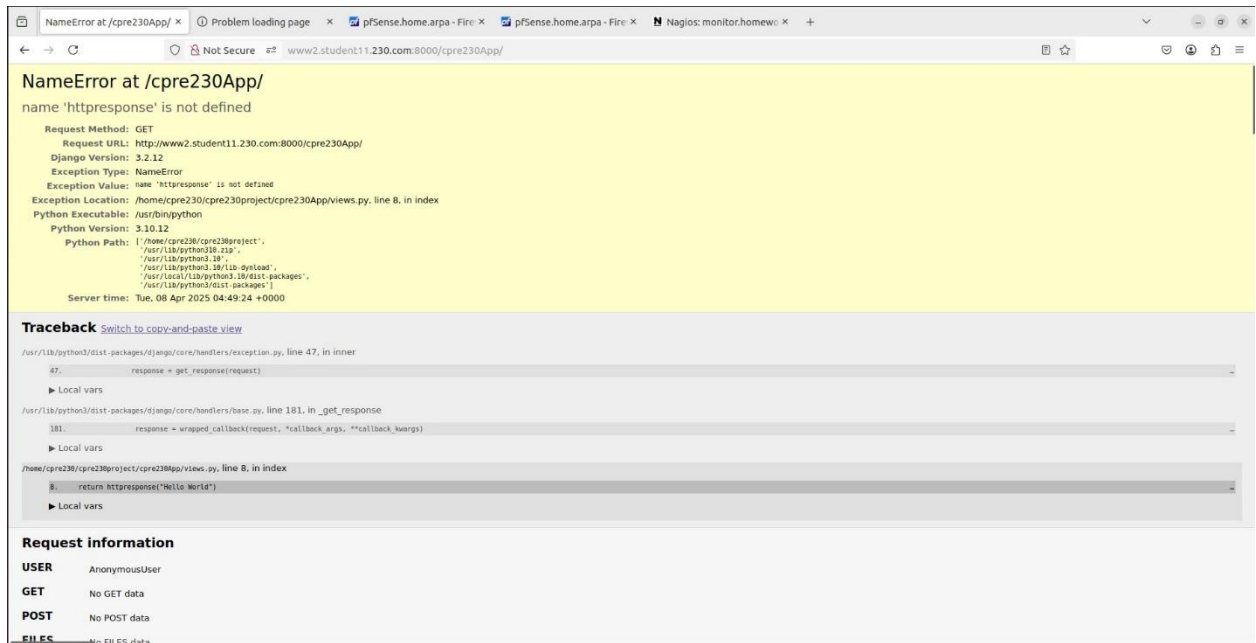


2.) Screenshot of testapp "Hello World" page (10 points)



3.) Screenshot of testapp debug information

- Identify 5 potentially dangerous facts revealed by the debug page.
 - How do you turn off the display of information?
- (10 points)



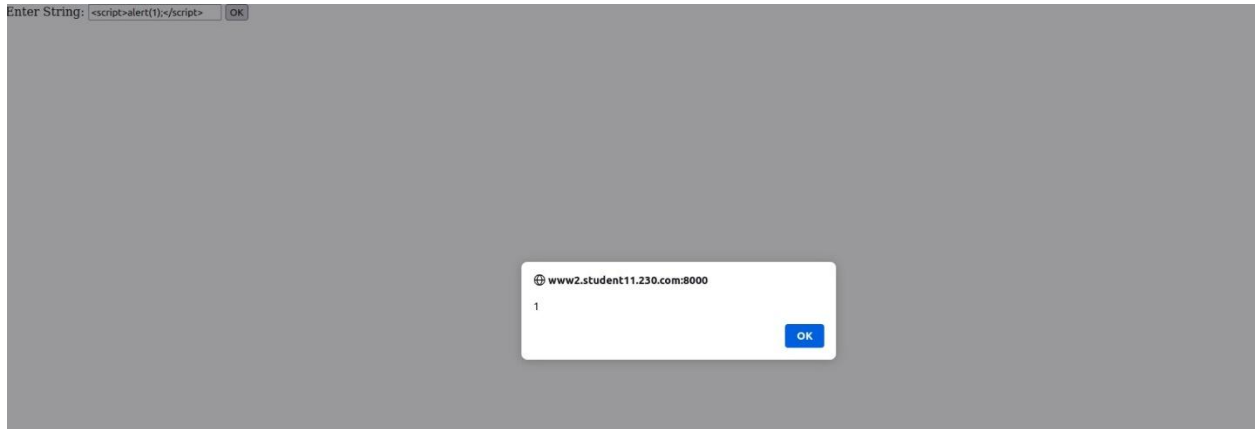
a) The 5 potential threats are:

- It displays exactly where the error is in the file and it also displays the file's complete path
- It reveals the logging info and web server files
- The system layout is based on the location of server from where it is running the machine.
- A secret key or some information related to a password hash might be exposed through some settings
- All environmental variables set like the proxies and no proxies

b) We can turn off the display information by setting `DEBUG = False` in `settings.py` file.

4.) Screenshot of XSS javascript alert

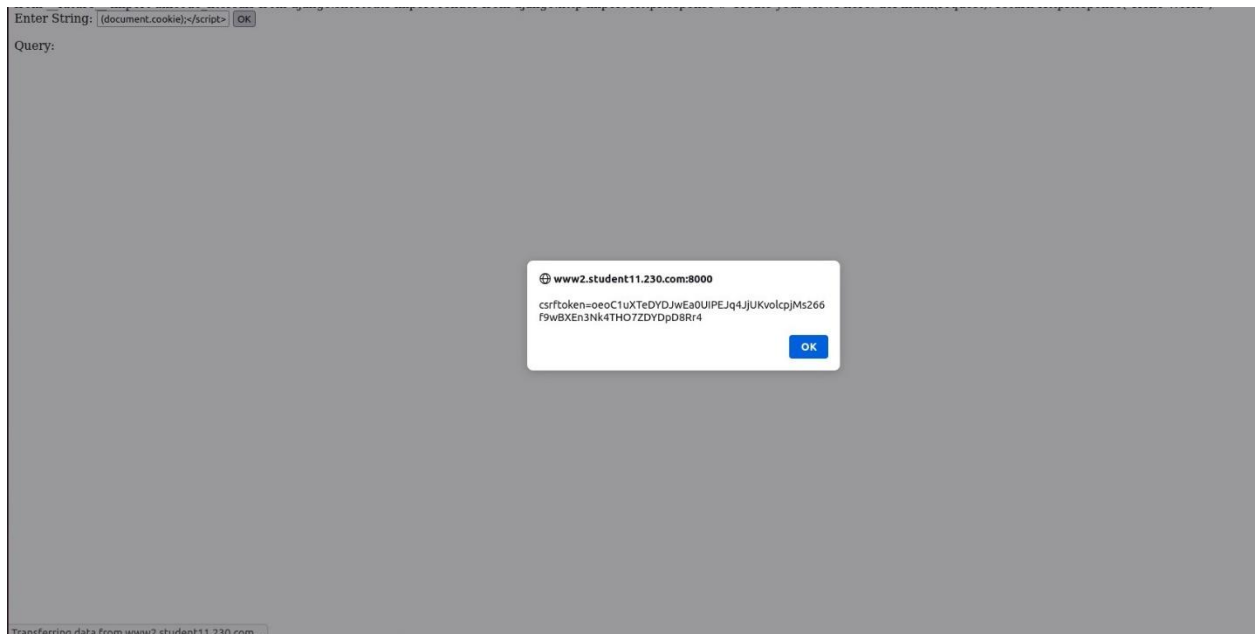
a.) Explain why the javascript isn't being displayed on the page
(10 points)



a) In this screenshot, the browser executes the script and shows a JavaScript alert box because the text is being handled by the html body, so it processes the command `<script>alert(1) </script>` as it is to mean that its interior is a script that needs to be run, which causes the javascript code to be run instead of just printing that text.

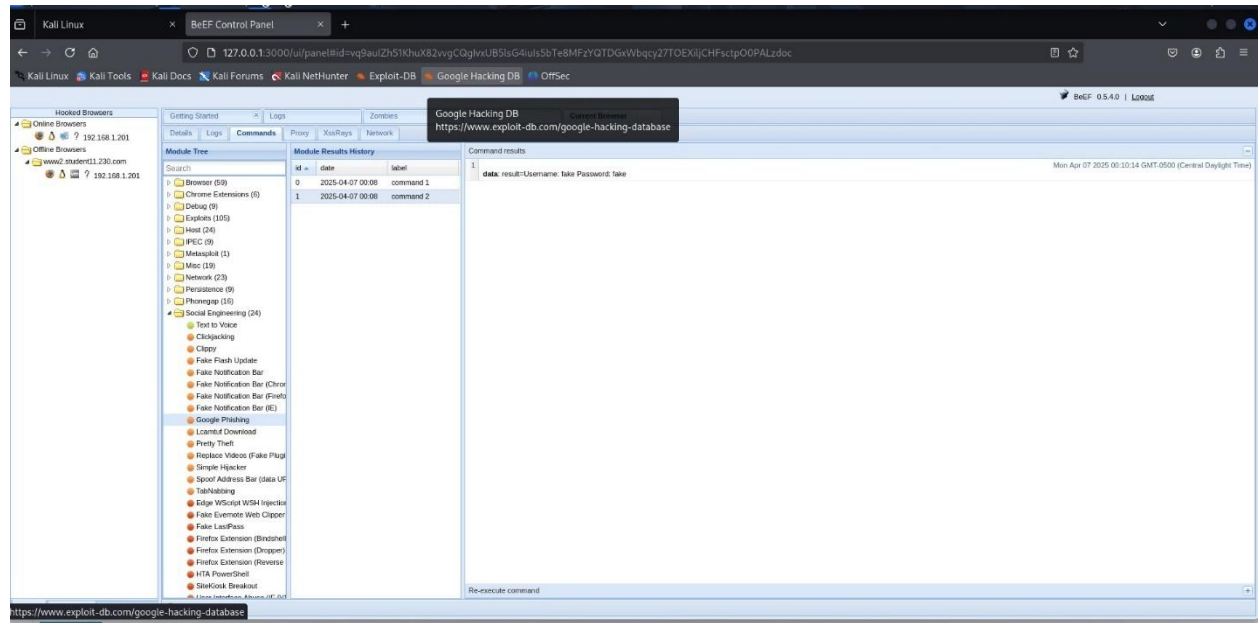
5.) Screenshot of alert(document.cookie)

a.) How did we get this value?
b.) Is it good or bad that we have that value?
(10 points)



- a) We get this value because the javascript code tells the system to retrieve the currently used cookie and print that out over here.
- b) It is very bad, since over here we get to know that an uncontrolled script could be used by a malicious to infiltrate this in a webpage to retrieve and use people's cookies.

6.) Screenshot of username/password captured with BeEF (10 points)



7.) Screenshot of fixed XSS page

- a.) Screenshot and explanation of the problem and how you fixed it. (10 points)



- a) Over here the vulnerability is caused by by the line `{% autoescape off%}` since it allows any malicious input like `<script>` to be executed in a browser, so even if we use `{{ query }}` over here, it doesn't make much sense since escaping is disabled over here. In order tom fix this issue we can set autoescape as on, so that when we run the command `<script>alert(1)</script>`, it will be shown as a text and will not be executed.

8.) Lookup the three types of XSS

a.) Which did we use in this lab?
(10 points)

Ans) 3 types of XSS are Stored XSS, Reflected XSS and DOM-Based XSS.

a) Firstly ,the script was not stored in any database. Secondly, the script was injected through a form and that's why we used Reflected XSS over here to correct xss_page.html.

9.) Screenshot of NAT rule (10 points)

Rules	Interface	Protocol	Source Address	Source Ports	Dest. Address	Dest. Ports	NAT IP	NAT Ports	Description	Actions
Desktop - Nagios check	WAN	TCP	*	*	42.49.30.201	22 (SSH)	192.168.1.201	22 (SSH)	desktop - Nagios ssh check	[Edit] [Delete]
NS2 - Nagios checks	WAN	TCP/UDP	37.56.23.100	*	42.49.30.199	53 (DNS)	192.168.1.200	53 (DNS)	ns2 - Nagios DNS for inside machines	[Edit] [Delete]
Idap-Nagios checks	WAN	TCP	37.56.23.100	*	42.49.30.205	389 (LDAP)	192.168.1.205	389 (LDAP)	Idap - Nagios checks	[Edit] [Delete]
Mail-Nagios checks	WAN	TCP	37.56.23.100	*	42.49.30.204	993 (IMAP/S)	192.168.1.204	993 (IMAP/S)	Allow IMAP access	[Edit] [Delete]
	WAN	TCP	*	*	42.49.30.204	587 (SUBMISSION)	192.168.1.204	587 (SUBMISSION)	Allow SMTP submission	[Edit] [Delete]
	WAN	TCP	*	*	42.49.30.204	110 (POP3)	192.168.1.204	110 (POP3)	Mail pop	[Edit] [Delete]
www - Nagios checks	WAN	TCP	*	*	42.49.30.202	80 (HTTP)	192.168.1.202	80 (HTTP)	Allow external web access to web server	[Edit] [Delete]
	WAN	TCP	*	*	42.49.30.202	443 (HTTPS)	192.168.1.202	443 (HTTPS)	Allow external web access to web server	[Edit] [Delete]
Django-Nagios checks	WAN	TCP	*	*	42.49.30.206	8000	192.168.1.206	8000	Django Access	[Edit] [Delete]

10.) Screenshot of Nagios with django services successfully running (10 points)

Student 11 (student11)

Host	Services	Actions
desktop1.student11.230.com	desktop1 ssh	[Monitor] [Refresh] [Details]
ldap.student11.230.com	ldap Bezos ldap Gates ldap Jobs ldap Musk ldap Zuckerberg ldap service	[Monitor] [Refresh] [Details]
mail.student11.230.com	dns MX mail pop secure imap secure imap login secure smtp	[Monitor] [Refresh] [Details]
ns1.student11.230.com	dns mail dns ns1 dns www dns www2 reverse dns mail reverse dns ns1 reverse dns www reverse dns www2	[Monitor] [Refresh] [Details]
ns2.student11.230.com	dns desktop1 dns ldap dns ws reverse dns desktop1 reverse dns ldap reverse dns ws	[Monitor] [Refresh] [Details]
www.student11.230.com	www http www https www login	[Monitor] [Refresh] [Details]
www2.student11.230.com	www2 django www2 ssh	[Monitor] [Refresh] [Details]